

PRACTICE EXERCISES OF THE MICROPROCESSORS & MICROCONTROLLERS

Instructor: The Tung Than

Student's name: Văn Nhật Tân

Student code: 24521587

PRACTICE EXERCISE #6

IO PROCESSING, CALCULATION AND MEMORY ON THE 8086 MICROPROCESSOR

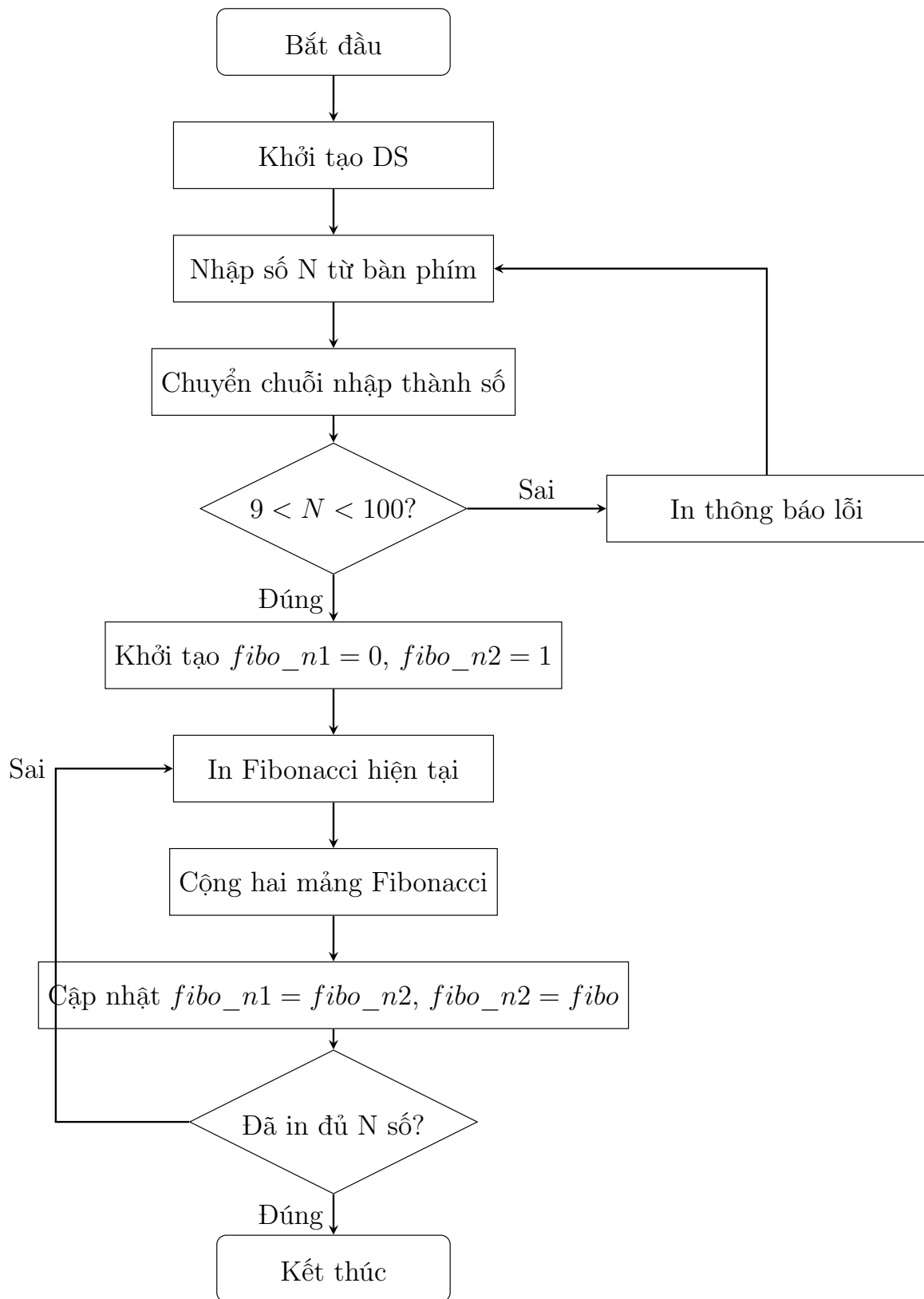
1 Yêu cầu bài thực hành

Bài thực hành yêu cầu viết chương trình Assembly 8086 thực hiện các chức năng sau:

- Nhập một số N có 2 chữ số từ bàn phím.
- Kiểm tra điều kiện: $9 < N < 100$.
- In ra màn hình N số Fibonacci đầu tiên.
- Thực hiện chương trình bằng hai cách khác nhau:
 - Phiên bản 1: sử dụng mảng để lưu trữ và tính toán.
 - Phiên bản 2: sử dụng các vùng nhớ riêng lẻ thay cho mảng.

2 Phiên bản 1: Chương trình sử dụng mảng

2.1 Flowchart chương trình sử dụng mảng



Hình 1: Flowchart chương trình Fibonacci sử dụng mảng

2.2 Nguyên lý hoạt động

Ở phiên bản sử dụng mảng, chương trình biểu diễn mỗi số Fibonacci bằng một mảng gồm 21 phần tử kiểu byte. Mỗi phần tử của mảng lưu một chữ số thập phân, do đó chương trình có thể xử lý được các số Fibonacci lớn hơn giới hạn 16-bit của vi xử lý 8086.

Ba mảng chính được sử dụng trong chương trình là:

<code>fibo_n1</code>	<code>DB 21 DUP(0)</code>
<code>fibo_n2</code>	<code>DB 21 DUP(0)</code>
<code>fibo</code>	<code>DB 21 DUP(0)</code>

Trong đó:

- `fibo_n1` lưu số Fibonacci hiện tại.
- `fibo_n2` lưu số Fibonacci kế tiếp.
- `fibo` lưu kết quả cộng của hai số trước đó.

Ban đầu, toàn bộ các phần tử của ba mảng đều có giá trị bằng 0. Chương trình gán phần tử cuối cùng của mảng `fibo_n2` bằng 1:

<code>MOV fibo_n2 + 20, 1</code>

Vì các chữ số được lưu từ trái sang phải, nên phần tử cuối cùng của mảng là hàng đơn vị. Khi đó:

$$\text{fibo_n1} = 0, \text{fibo_n2} = 1$$

Đây là hai giá trị đầu tiên để tạo dãy Fibonacci.

Sau khi nhập số N từ bàn phím, chương trình dùng hàm `ConvertStrToNum` để chuyển chuỗi ký tự nhập vào thành số. Ví dụ, nếu người dùng nhập chuỗi “20”, chương trình sẽ xử lý từng ký tự, nhân kết quả hiện tại với 10 rồi cộng thêm chữ số mới. Kết quả cuối cùng được lưu vào biến `Num`.

Tiếp theo, chương trình kiểm tra điều kiện:

<code>CMP AL, 9</code>
<code>JLE InvalidInput</code>
<code>CMP AL, 100</code>
<code>JGE InvalidInput</code>

Nếu N nhỏ hơn hoặc bằng 9, hoặc N lớn hơn hoặc bằng 100, chương trình sẽ in thông báo lỗi và yêu cầu nhập lại. Nếu N hợp lệ, chương trình bắt đầu in dãy Fibonacci.

Trong mỗi vòng lặp, chương trình thực hiện bốn bước chính:

1. In giá trị hiện tại của `fibo_n1`.
2. Tính `fibo = fibo_n1 + fibo_n2`.
3. Sao chép `fibo_n2` sang `fibo_n1`.

4. Sao chép fibo sang fibo_n2.

Việc cộng hai số Fibonacci được thực hiện trong thủ tục `Add_Array`. Chương trình cộng từ phần tử cuối mảng về đầu mảng, giống như cách cộng hai số thập phân bằng tay. Biến `SI` được dùng làm chỉ số mảng và bắt đầu từ vị trí 20, tức là chữ số hàng đơn vị:

```
MOV SI, 20
MOV DL, 0
```

Biến `DL` được dùng để lưu phần nhớ. Ở mỗi bước, chương trình lấy một chữ số của `fibo_n1`, một chữ số của `fibo_n2`, cộng thêm phần nhớ trước đó rồi lưu kết quả vào mảng `fibo`.

Nếu tổng nhỏ hơn 10, chữ số kết quả được lưu trực tiếp. Nếu tổng lớn hơn hoặc bằng 10, chương trình trừ đi 10 và đặt `DL = 1` để nhớ sang chữ số tiếp theo.

Sau khi cộng xong, chương trình cập nhật dãy Fibonacci bằng thủ tục `copyArray`. Vì trong dãy Fibonacci, số tiếp theo được tính theo công thức:

$$F(n) = F(n - 1) + F(n - 2)$$

nên sau mỗi lần tính, cần cập nhật:

```
fibo_n1 = fibo_n2
fibo_n2 = fibo
```

Thủ tục `printFibo` dùng để in số Fibonacci ra màn hình. Vì mỗi mảng có 21 phần tử nên các số nhỏ sẽ có nhiều số 0 ở đầu. Do đó chương trình dùng thanh ghi `DH` để đánh dấu đã gặp chữ số khác 0 hay chưa. Nếu chưa gặp chữ số khác 0, các số 0 ở đầu sẽ bị bỏ qua. Khi gặp chữ số khác 0 đầu tiên, chương trình bắt đầu in các chữ số còn lại.

Nếu toàn bộ mảng đều bằng 0, chương trình sẽ in ra ký tự '0'. Điều này giúp số Fibonacci đầu tiên được hiển thị đúng là 0.

2.3 Source code phiên bản sử dụng mảng

```
.MODEL SMALL
.STACK 100h

.DATA
    Input_mgs    DB "Enter number: $"
    Output_mgs    DB 13,10,"The first N Fibonacci numbers:",13,10,"$"
    Error_mgs     DB 13,10,"Invalid N! Please enter 9 < N < 100.",13,10,"$"
    Space_mgs     DB " $"

    input1        DB 3,?,3 DUP(' ')
    Num            DB ?

    fibo_n1        DB 21 DUP(0)
```

```

    fibo_n2      DB 21 DUP(0)
    fibo         DB 21 DUP(0)

.CODE

Main PROC
    MOV AX, @DATA
    MOV DS, AX

InputAgain:
    LEA BX, Input_mgs
    LEA CX, input1
    CALL Input
    CALL Endline

    LEA SI, input1 + 2
    LEA BX, Num
    CALL ConvertStrToNum

    MOV AL, Num
    CMP AL, 9
    JLE InvalidInput
    CMP AL, 100
    JGE InvalidInput
    JMP StartProgram

InvalidInput:
    LEA DX, Error_mgs
    MOV AH, 9
    INT 21h
    JMP InputAgain

StartProgram:
    LEA DX, Output_mgs
    MOV AH, 9
    INT 21h

    MOV fibo_n2 + 20, 1

    XOR CX, CX
    MOV CL, Num

PrintLoop:
    LEA BX, fibo_n1
    CALL printFibo

```

```

    LEA DX, Space_mgs
    MOV AH, 9
    INT 21h

    CALL Add_Array

    LEA AX, fibo_n1
    LEA DX, fibo_n2
    CALL copyArray

    LEA AX, fibo_n2
    LEA DX, fibo
    CALL copyArray

    LOOP PrintLoop

    CALL Endline

    MOV AH, 4Ch
    INT 21h
Main ENDP

printFibo PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX
    PUSH SI

    MOV SI, 0
    MOV CX, 21
    XOR DH, DH

loop_printFibo:
    MOV DL, [BX + SI]

    CMP DL, 0
    JNE print_digit

    CMP DH, 0
    JE skip_zero

print_digit:
    MOV DH, 1

```

```

    ADD DL, '0'
    MOV AH, 02h
    INT 21h

skip_zero:
    INC SI
    LOOP loop_printFibo

    CMP DH, 1
    JE end_printFibo

    MOV DL, '0'
    MOV AH, 02h
    INT 21h

end_printFibo:
    POP SI
    POP DX
    POP CX
    POP BX
    POP AX
    RET
printFibo ENDP

copyArray PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH SI
    PUSH DI

    MOV DI, AX
    MOV SI, DX
    MOV CX, 21

copy_loop:
    MOV BL, [SI]
    MOV [DI], BL
    INC SI
    INC DI
    LOOP copy_loop

    POP DI
    POP SI
    POP CX

```

```

    POP BX
    POP AX
    RET
copyArray ENDP

Add_Array PROC
    PUSH AX
    PUSH BX
    PUSH DX
    PUSH SI

    MOV SI, 20
    MOV DL, 0

loop_add_array:
    CMP SI, 0
    JL end_loop_add_array

    MOV AL, fibo_n1[SI]
    MOV BL, fibo_n2[SI]

    ADD AL, BL
    ADD AL, DL

    MOV DL, 0
    CMP AL, 10
    JL store_digit

    SUB AL, 10
    MOV DL, 1

store_digit:
    MOV fibo[SI], AL

    DEC SI
    JMP loop_add_array

end_loop_add_array:
    POP SI
    POP DX
    POP BX
    POP AX
    RET
Add_Array ENDP

```


ConvertStrToNum PROC

PUSH AX

PUSH BX

PUSH CX

PUSH SI

XOR AX, AX

Convert_loop:

MOV CL, [SI]

CMP CL, '\$'

JE end_loop

MOV CH, 0

PUSH BX

MOV BX, 10

MUL BX

SUB CX, '0'

ADD AX, CX

POP BX

INC SI

JMP Convert_loop

end_loop:

MOV [BX], AL

POP SI

POP CX

POP BX

POP AX

RET

ConvertStrToNum ENDP

Input PROC

PUSH AX

PUSH BX

PUSH CX

PUSH DX

PUSH SI

MOV DX, BX

```

MOV AH, 9
INT 21h

MOV DX, CX
MOV AH, 0Ah
INT 21h

MOV BX, CX
XOR SI, SI
MOV AL, [BX + 1]
MOV AH, 0
MOV SI, AX
MOV BYTE PTR [BX + SI + 2], '$'

POP SI
POP DX
POP CX
POP BX
POP AX
RET
Input ENDP

Endline PROC
PUSH AX
PUSH DX

MOV DL, 13
MOV AH, 2
INT 21h

MOV DL, 10
MOV AH, 2
INT 21h

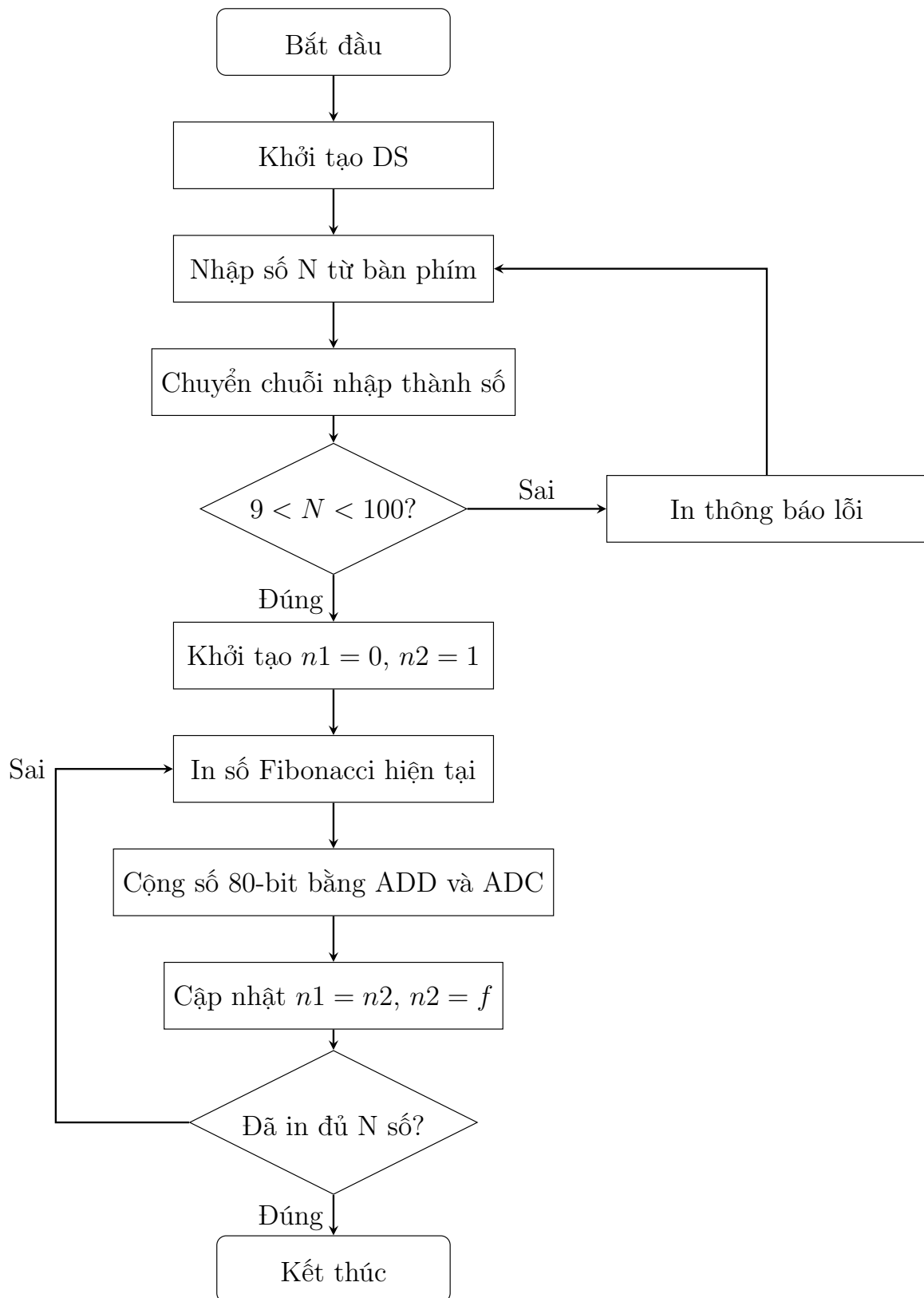
POP DX
POP AX
RET
Endline ENDP

END Main

```

3 Phiên bản 2: Chương trình sử dụng memory

3.1 Flowchart chương trình sử dụng memory



Hình 2: Flowchart chương trình Fibonacci sử dụng memory

3.2 Nguyên lý hoạt động

Mỗi số Fibonacci được lưu bằng nhiều biến WORD riêng lẻ. Mỗi biến WORD có kích thước 16-bit, vì vậy chương trình ghép 5 WORD lại để biểu diễn một số lớn 80-bit (Vì số fibonacci thứ 99 có độ lớn 68 bit).

Các biến dùng để lưu số Fibonacci hiện tại là:

```
n1_0 DW 0 //16bit thập nhất.  
n1_1 DW 0  
n1_2 DW 0  
n1_3 DW 0  
n1_4 DW 0
```

Các biến dùng để lưu số Fibonacci kế tiếp là:

```
n2_0 DW 1 //16bit thập nhất.  
n2_1 DW 0  
n2_2 DW 0  
n2_3 DW 0  
n2_4 DW 0
```

Các biến dùng để lưu kết quả cộng là:

```
f_0 DW 0 //16bit thập nhất.  
f_1 DW 0  
f_2 DW 0  
f_3 DW 0  
f_4 DW 0
```

Tổng dung lượng dùng để lưu một số là:

$$5 \text{ WORD} = 5 \times 16 = 80 \text{ bit}$$

Dung lượng 80-bit đủ để lưu các số Fibonacci trong phạm vi yêu cầu của đề bài, vì N nhỏ hơn 100. Các số Fibonacci trong phạm vi này lớn hơn 16-bit, nhưng vẫn nhỏ hơn khả năng lưu trữ của 80-bit.

Sau khi nhập số N, chương trình cũng dùng thủ tục `ConvertStrToNum` để chuyển chuỗi nhập từ bàn phím thành số nguyên và lưu vào biến `Num`. Sau đó chương trình kiểm tra điều kiện:

$$9 < N < 100$$

Nếu N không hợp lệ, chương trình in thông báo lỗi và yêu cầu nhập lại. Nếu N hợp lệ, chương trình khởi tạo:

$$n1 = 0, n2 = 1$$

Sau đó bắt đầu vòng lặp in dãy Fibonacci.

Trong mỗi vòng lặp, chương trình thực hiện các bước:

1. In giá trị hiện tại của **n1**.
2. Tính $f = n1 + n2$.
3. Cập nhật $n1 = n2$.
4. Cập nhật $n2 = f$.
5. Giảm bộ đếm và kiểm tra đã in đủ **N** số hay chưa.

Phép cộng hai số Fibonacci được thực hiện trong thủ tục **AddFibo**. Do 8086 chỉ xử lý trực tiếp dữ liệu 16-bit, chương trình phải cộng từng WORD từ thấp đến cao.

Đầu tiên, chương trình cộng 16 bit thấp nhất bằng lệnh **ADD**:

```
MOV AX, n1_0
ADD AX, n2_0
MOV f_0, AX
```

Nếu phép cộng phần thấp bị tràn, cờ Carry Flag sẽ được bật. Khi cộng các WORD tiếp theo, chương trình dùng lệnh **ADC**.

Nhờ vậy, phần nhớ từ WORD thấp sẽ được chuyển sang WORD cao hơn. Cách làm này tương tự cộng nhiều chữ số, nhưng thay vì cộng từng chữ số thập phân, chương trình cộng từng khối 16-bit.

Sau khi tính được kết quả **f**, chương trình gọi thủ tục **UpdateFibo** để cập nhật giá trị cho lần lặp tiếp theo:

$$\begin{aligned} n1 &= n2 \\ n2 &= f \end{aligned}$$

Để in kết quả ra dưới dạng số thập phân, chương trình sao chép giá trị của **n1** sang các biến tạm:

```
t_0, t_1, t_2, t_3, t_4
```

Sau đó chương trình liên tục chia số 80-bit này cho 10. Mỗi lần chia, phần dư chính là một chữ số thập phân của số cần in. Tuy nhiên, các chữ số dư thu được theo thứ tự từ phải sang trái, tức là từ hàng đơn vị đến hàng cao hơn. Vì vậy, chương trình đưa các chữ số này vào stack bằng lệnh **PUSH**, sau đó lấy ra bằng lệnh **POP** để in theo đúng thứ tự.

Thủ tục **DivTempBy10** thực hiện phép chia số 80-bit cho 10. Chương trình chia từ WORD cao nhất xuống WORD thấp nhất. Phần dư của lần chia trước được giữ trong thanh ghi **DX** và tiếp tục tham gia vào lần chia kế tiếp. Nhờ vậy, toàn bộ 5 WORD được xem như một số 80-bit thống nhất.

Nếu số cần in bằng 0, chương trình không thực hiện quá trình chia mà in trực tiếp ký tự '0' ra màn hình.

3.3 Source code phiên bản sử dụng memory

```
.MODEL SMALL
.STACK 100h

.DATA
    Input_mgs    DB "Enter a 2-digits number: $"
    Output_mgs   DB 13,10,"The first N Fibonacci numbers:",13,10,"$"
    Error_mgs    DB 13,10,"Invalid N! Please enter 9 < N < 100.",13,10,"$"
    Space_mgs    DB " $"

    input1       DB 3,?,3 DUP(' ')
    Num           DB ?

    n1_0 DW 0
    n1_1 DW 0
    n1_2 DW 0
    n1_3 DW 0
    n1_4 DW 0

    n2_0 DW 1
    n2_1 DW 0
    n2_2 DW 0
    n2_3 DW 0
    n2_4 DW 0

    f_0 DW 0
    f_1 DW 0
    f_2 DW 0
    f_3 DW 0
    f_4 DW 0

    t_0 DW 0
    t_1 DW 0
    t_2 DW 0
    t_3 DW 0
    t_4 DW 0

    Ten DW 10

.CODE

Main PROC
    MOV AX, @DATA
    MOV DS, AX
```

InputAgain:

```
    LEA BX, Input_mgs
    LEA CX, input1
    CALL Input
    CALL Endline

    LEA SI, input1 + 2
    LEA BX, Num
    CALL ConvertStrToNum

    MOV AL, Num
    CMP AL, 9
    JLE InvalidInput
    CMP AL, 100
    JGE InvalidInput
    JMP StartProgram
```

InvalidInput:

```
    LEA DX, Error_mgs
    MOV AH, 9
    INT 21h
    JMP InputAgain
```

StartProgram:

```
    MOV n1_0, 0
    MOV n1_1, 0
    MOV n1_2, 0
    MOV n1_3, 0
    MOV n1_4, 0

    MOV n2_0, 1
    MOV n2_1, 0
    MOV n2_2, 0
    MOV n2_3, 0
    MOV n2_4, 0

    LEA DX, Output_mgs
    MOV AH, 9
    INT 21h

    XOR CX, CX
    MOV CL, Num
```

PrintLoop:

```

CALL PrintN1

LEA DX, Space_mgs
MOV AH, 9
INT 21h

CALL AddFibo
CALL UpdateFibo

LOOP PrintLoop

CALL Endline

MOV AH, 4Ch
INT 21h
Main ENDP

AddFibo PROC
    PUSH AX

    MOV AX, n1_0
    ADD AX, n2_0
    MOV f_0, AX

    MOV AX, n1_1
    ADC AX, n2_1
    MOV f_1, AX

    MOV AX, n1_2
    ADC AX, n2_2
    MOV f_2, AX

    MOV AX, n1_3
    ADC AX, n2_3
    MOV f_3, AX

    MOV AX, n1_4
    ADC AX, n2_4
    MOV f_4, AX

    POP AX
    RET
AddFibo ENDP

UpdateFibo PROC

```



```

    PUSH AX

    MOV AX, n2_0
    MOV n1_0, AX
    MOV AX, n2_1
    MOV n1_1, AX
    MOV AX, n2_2
    MOV n1_2, AX
    MOV AX, n2_3
    MOV n1_3, AX
    MOV AX, n2_4
    MOV n1_4, AX

    MOV AX, f_0
    MOV n2_0, AX
    MOV AX, f_1
    MOV n2_1, AX
    MOV AX, f_2
    MOV n2_2, AX
    MOV AX, f_3
    MOV n2_3, AX
    MOV AX, f_4
    MOV n2_4, AX

    POP AX
    RET
UpdateFibo ENDP

PrintN1 PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX

    MOV AX, n1_0
    MOV t_0, AX
    MOV AX, n1_1
    MOV t_1, AX
    MOV AX, n1_2
    MOV t_2, AX
    MOV AX, n1_3
    MOV t_3, AX
    MOV AX, n1_4
    MOV t_4, AX

```

```

    CMP t_0, 0
    JNE ConvertPrint
    CMP t_1, 0
    JNE ConvertPrint
    CMP t_2, 0
    JNE ConvertPrint
    CMP t_3, 0
    JNE ConvertPrint
    CMP t_4, 0
    JNE ConvertPrint

    MOV DL, '0'
    MOV AH, 2
    INT 21h
    JMP EndPrintN1

ConvertPrint:
    XOR CX, CX

DivideLoop:
    CALL DivTempBy10
    ADD DL, '0'
    MOV DH, 0
    PUSH DX
    INC CX

    CMP t_0, 0
    JNE DivideLoop
    CMP t_1, 0
    JNE DivideLoop
    CMP t_2, 0
    JNE DivideLoop
    CMP t_3, 0
    JNE DivideLoop
    CMP t_4, 0
    JNE DivideLoop

PrintDigitLoop:
    POP DX
    MOV AH, 2
    INT 21h
    LOOP PrintDigitLoop

EndPrintN1:
    POP DX

```

```

    POP CX
    POP BX
    POP AX
    RET
PrintN1 ENDP

DivTempBy10 PROC
    PUSH AX
    PUSH BX

    MOV BX, Ten
    XOR DX, DX

    MOV AX, t_4
    DIV BX
    MOV t_4, AX

    MOV AX, t_3
    DIV BX
    MOV t_3, AX

    MOV AX, t_2
    DIV BX
    MOV t_2, AX

    MOV AX, t_1
    DIV BX
    MOV t_1, AX

    MOV AX, t_0
    DIV BX
    MOV t_0, AX

    POP BX
    POP AX
    RET
DivTempBy10 ENDP

ConvertStrToNum PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH SI

    XOR AX, AX

```

```

Convert_loop:
    MOV CL, [SI]
    CMP CL, '$'
    JE end_loop

    MOV CH, 0
    PUSH BX

    MOV BX, 10
    MUL BX

    SUB CX, '0'
    ADD AX, CX

    POP BX

    INC SI
    JMP Convert_loop

end_loop:
    MOV [BX], AL

    POP SI
    POP CX
    POP BX
    POP AX
    RET
ConvertStrToNum ENDP

Input PROC
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX
    PUSH SI

    MOV DX, BX
    MOV AH, 9
    INT 21h

    MOV DX, CX
    MOV AH, 0Ah
    INT 21h

```

```

MOV BX, CX
XOR SI, SI
MOV AL, [BX + 1]
MOV AH, 0
MOV SI, AX
MOV BYTE PTR [BX + SI + 2], '$'

POP SI
POP DX
POP CX
POP BX
POP AX
RET
Input ENDP

Endline PROC
PUSH AX
PUSH DX

MOV DL, 13
MOV AH, 2
INT 21h

MOV DL, 10
MOV AH, 2
INT 21h

POP DX
POP AX
RET
Endline ENDP

END Main

```

4 So sánh hai phiên bản chương trình

Tiêu chí	Phiên bản dùng mảng	Phiên bản dùng memory
Cách lưu trữ	Mỗi số Fibonacci được lưu bằng một mảng gồm 21 phần tử.	Mỗi số Fibonacci được lưu bằng 5 biến WORD riêng lẻ.
Đơn vị lưu trữ	Mỗi phần tử lưu một chữ số thập phân.	Mỗi WORD lưu 16 bit dữ liệu nhị phân.

Cách tính toán	Cộng từng chữ số từ phải sang trái.	Cộng từng WORD từ thấp đến cao.
Xử lý nhớ	Tự xử lý nhớ bằng biến DL.	Dùng Carry Flag thông qua lệnh ADD và ADC.
Cách in kết quả	In trực tiếp từng chữ số trong mảng.	Phải chia số 80-bit nhiều lần cho 10 để chuyển sang thập phân.
Tốc độ tính toán	Chậm hơn vì xử lý từng chữ số.	Nhanh hơn vì xử lý theo từng khối 16 bit.
Khả năng mở rộng	Dễ mở rộng bằng cách tăng kích thước mảng.	Muốn mở rộng phải khai báo thêm các biến WORD.

4.1 Nhận xét

Phiên bản sử dụng mảng dễ triển khai và mở rộng vì mỗi phần tử của mảng lưu một chữ số thập phân. Khi in ra màn hình, chương trình chỉ cần bỏ qua các số 0 ở đầu và in từng chữ số còn lại. Tuy nhiên, tốc độ tính toán không cao vì chương trình phải cộng từng chữ số một.

Phiên bản sử dụng memory xử lý hiệu quả hơn trong phần tính toán vì mỗi lần cộng được thực hiện trên 16 bit. Chương trình tận dụng được các lệnh ADD và ADC của 8086 để xử lý số lớn. Tuy nhiên, việc in kết quả ra dạng thập phân phức tạp hơn vì số được lưu dưới dạng nhị phân, cần chia nhiều lần cho 10 để chuyển đổi sang chữ số thập phân.

Tuy nhiên có thể thấy tốc độ xử lý khi sử dụng memory nhanh hơn nhiều so với xử lý bằng mảng.

5 Kết quả mô phỏng

Khi nhập:

N = 10

Kết quả chương trình hiển thị:

The first N Fibonacci numbers:
0 1 1 2 3 5 8 13 21 34

Khi nhập:

N = 20

Kết quả chương trình hiển thị:

The first N Fibonacci numbers:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181

6 Video mô phỏng chương trình và source code

Link Google Drive:

https://drive.google.com/drive/folders/1NktBn-_n3oPgnMUOpyoJI03ZfNn4nY1j?usp=drive_link